REACT

MODULE 3

State, Hooks & Interactive Applications

useState • useEffect • useRef • useContext • API Data • Projects



AI-generated visual reference • classroom notes

Detailed step-by-step notes with diagrams, multiple cases, exercises and solutions

Module 3 Overview

Module 1 taught components and props. Module 2 taught styling, layouts, events and forms. Module 3 makes those interfaces genuinely dynamic by teaching state, Hooks and data flow.

Roadmap

1. **State & Hooks** • Remember information and re-render UI
2. **useState** • Primitive, array and object state
3. **State Updates** • Functional updates and immutable changes
4. **useEffect** • Timers, browser APIs and synchronization
5. **Derived Data** • Calculate instead of duplicating state
6. **Lifting State** • Share state between components
7. **useRef** • DOM references and non-rendering values
8. **useContext** • Deeply shared application data
9. **API Data** • Loading, success, error and empty states
10. **Custom Hooks** • Reuse stateful logic
11. **Projects** • Todo + API dashboard + final assessment

Prerequisites

Students should already understand JSX, components, props, map(), events, forms, CSS and basic JavaScript arrays/objects.

MODULE 3 • PART 1

State Fundamentals

1.1 What is state?

State is information a component remembers between renders. Updating state through its setter asks React to render the component again with the new information.

User action → state update → new UI



1.2 Normal variable vs state

Why this does not behave like a React counter

```
function Counter() {
  let count = 0

  function handleClick() {
    count = count + 1
  }

  return (
    <button onClick={handleClick}>
      Count: {count}
    </button>
  )
}
```

Correct version with state

```
import { useState } from "react"

function Counter() {
  const [count, setCount] = useState(0)

  function handleClick() {
    setCount(count + 1)
  }

  return (
    <button onClick={handleClick}>
      Count: {count}
    </button>
  )
}
```

1.3 Hooks

Hooks are functions that let function components use React features. Module 3 concentrates on state, effects, refs and context. React's official reference groups these as built-in Hooks. ■cite■turn0search3■

Rules to teach early

- Call Hooks at the top level of your component.
- Do not call Hooks inside loops or conditions.
- Do not call Hooks inside event handlers.
- Keep Hook calls in a consistent order.

Hook placement

```
// ■ Correct
function Profile() {
  const [name, setName] = useState("Riya")
  return <h1>{name}</h1>
}

// ■ Incorrect
function Profile({ loggedIn }) {
  if (loggedIn) {
    const [name, setName] = useState("Riya")
  }

  return <h1>Profile</h1>
}
```

Mental model

A Hook is not a special variable. It is a React API that participates in the component's rendering lifecycle.

MODULE 3 • PART 2

useState: Different Types of State

2.1 Syntax

General form

```
const [value, setValue] = useState(initialValue)
```

2.2 String state

String

```
function NameEditor() {
  const [name, setName] = useState("")

  return (
    <div>
      <input
        value={name}
        onChange={(event) =>
          setName(event.target.value)
        }
      />

      <p>Hello {name || "Guest"}</p>
    </div>
  )
}
```

2.3 Boolean state

Boolean

```
function Menu() {
  const [open, setOpen] = useState(false)

  return (
    <>
      <button onClick={() => setOpen(!open)}>
        {open ? "Close" : "Open"}
      </button>

      {open && <nav>Menu content</nav>}
    </>
  )
}
```

2.4 Number state

Quantity

```
function Quantity() {
  const [quantity, setQuantity] = useState(1)

  return (
    <div>
      <button
        onClick={() =>
          setQuantity((q) => Math.max(1, q - 1))
        }
      >
        -
      </button>

      <span>{quantity}</span>

      <button
        onClick={() =>
          setQuantity((q) => q + 1)
        }
      >
        +
      </button>
    </div>
  )
```

Quantity (continued)

```
}
```

2.5 Array state

Add to array

```
function ShoppingList() {
  const [items, setItems] = useState([
    "Milk",
    "Bread"
  ])

  function addItem() {
    setItems((current) => [
      ...current,
      "Eggs"
    ])
  }

  return (
    <ul>
      {items.map((item) => (
        <li key={item}>{item}</li>
      ))}
      <button onClick={addItem}>Add</button>
    </ul>
  )
}
```

2.6 Remove from array

Immutable removal

```
function removeItem(id) {
  setItems((current) =>
    current.filter((item) => item.id !== id)
  )
}
```

2.7 Update an array item

Immutable update

```
function toggleItem(id) {
  setItems((current) =>
    current.map((item) =>
      item.id === id
        ? { ...item, completed: !item.completed }
        : item
    )
  )
}
```

2.8 Object state

Object update

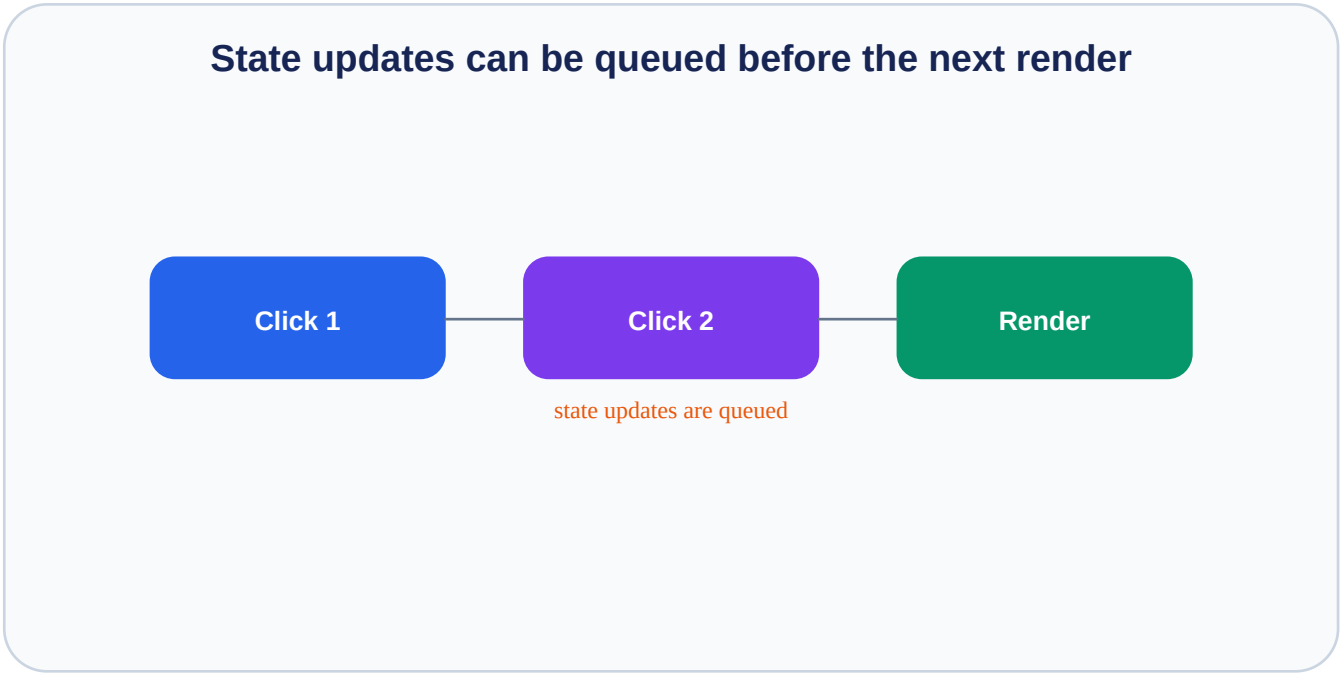
```
const [user, setUser] = useState({
  name: "Riya",
  age: 20,
  city: "Kaithal"
})

function changeCity() {
  setUser((current) => ({
    ...current,
    city: "Delhi"
  }))
}
```

Never mutate state directly

Avoid push(), splice(), object.property = value and similar mutations on state. Create a new array/object and pass it to the setter.

2.9 Functional updater



Three direct updates

```
function handleClick() {
  setCount(count + 1)
  setCount(count + 1)
  setCount(count + 1)
}
```

Three functional updates

```
function handleClick() {
  setCount((current) => current + 1)
  setCount((current) => current + 1)
  setCount((current) => current + 1)
}
```

Use the functional updater when the next value depends on the previous value.

2.10 Nested objects

Nested immutable update

```
const [profile, setProfile] = useState({
  name: "Riya",
  address: {
    city: "Kaithal",
    pin: "136027"
  }
})

function changeCity() {
  setProfile((current) => ({
    ...current,
    address: {
      ...current.address,
      city: "Delhi"
    }
  }))
}
```

Exercises: useState

Exercise 1: Counter

Level: Beginner+

- Start at 0.
- Add Increment, Decrement and Reset.
- Do not allow values below 0.

Exercise 2: Theme Toggle

Level: Beginner+

- Store light/dark in state.
- Toggle the state from a button.
- Change the UI class based on the state.

Exercise 3: Student Editor

Level: Intermediate

- Store name, age and city in one object.
- Create controlled inputs.
- Update only the changed property.
- Display the live result.

Exercise 4: Shopping Cart

Level: Intermediate

- Store cart items in an array.
- Add items.
- Remove items.
- Toggle a selected item.

Solutions: useState

Solution 1: Counter

Complete solution

```
import { useState } from "react"

function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <h1>{count}</h1>

      <button
        onClick={() =>
          setCount((c) => c + 1)
        }
      >
        Increment
      </button>

      <button
        onClick={() =>
          setCount((c) => Math.max(0, c - 1))
        }
      >
        Decrement
      </button>
```

Complete solution (continued)

```
      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  )
}

export default Counter
```

Solution 2: Theme

Complete solution

```
function ThemeToggle() {
  const [dark, setDark] = useState(false)

  return (
    <main className={
      dark ? "page dark" : "page light"
    }>
      <h1>{dark ? "Dark" : "Light"}</h1>

      <button onClick={() => setDark((d) => !d)}>
        Toggle
      </button>
    </main>
  )
}
```

Solutions continued

Solution 3: Student Editor

Solution

```
function StudentEditor() {
  const [student, setStudent] = useState({
    name: "",
    age: "",
    city: ""
  })

  function handleChange(event) {
    const { name, value } = event.target

    setStudent((current) => ({
      ...current,
      [name]: value
    }))
  }

  return (
    <div>
      <input
        name="name"
        value={student.name}
        onChange={handleChange}
        placeholder="Name"
      />
```

Solution (continued)

```
      <input
        name="age"
        value={student.age}
        onChange={handleChange}
        placeholder="Age"
      />

      <input
        name="city"
        value={student.city}
        onChange={handleChange}
        placeholder="City"
      />

      <h2>{student.name || "Student"}</h2>
      <p>{student.age}</p>
      <p>{student.city}</p>
    </div>
  )
}
```

Solution 4: Shopping Cart

Solution

```
function Cart() {
  const [items, setItems] = useState([])

  function add(item) {
    setItems((current) => [...current, item])
  }

  function remove(id) {
    setItems((current) =>
      current.filter((item) => item.id !== id)
    )
  }

  function toggle(id) {
    setItems((current) =>
      current.map((item) =>
        item.id === id
          ? { ...item, selected: !item.selected }
          : item
      )
    )
  }

  return (
```

Solution (continued)

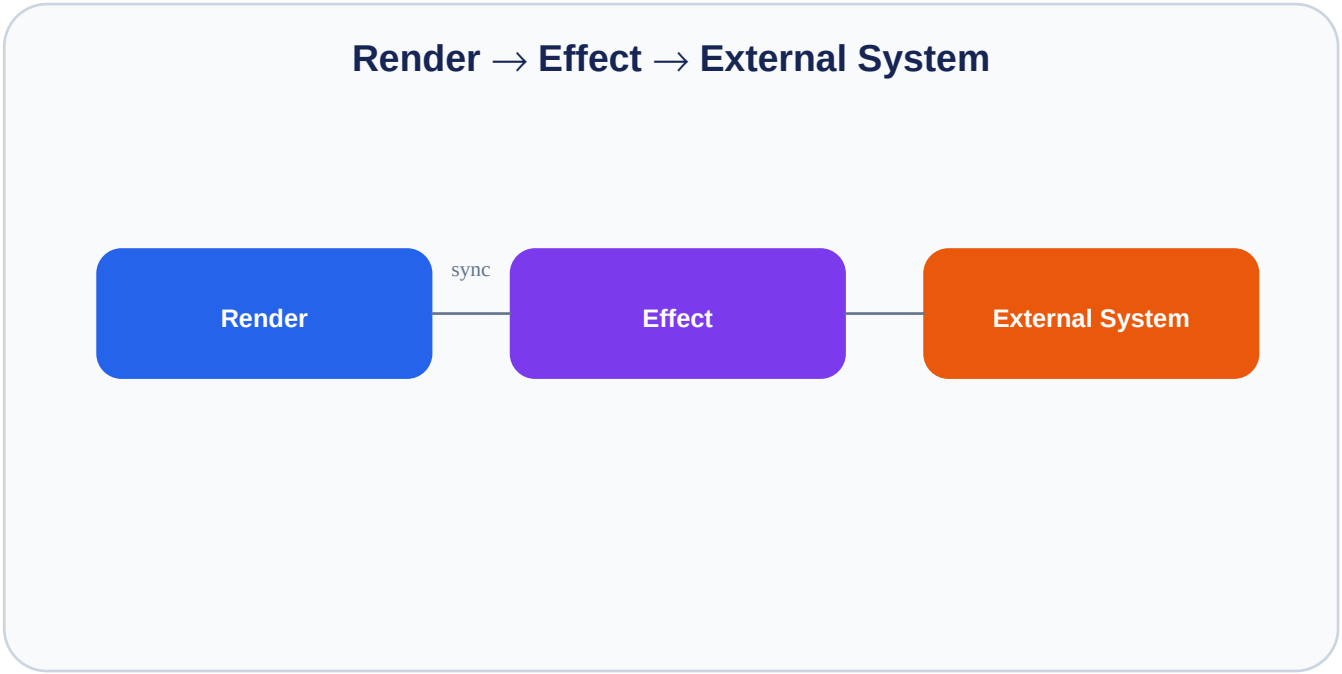
```
    <CartList
      items={items}
      onAdd={add}
      onRemove={remove}
      onToggle={toggle}
    />
  )
}
```

MODULE 3 • PART 3

useEffect: Synchronizing with External Systems

3.1 What is an Effect?

React's current documentation defines useEffect as a Hook for synchronizing a component with an external system. Examples include timers, browser APIs, network connections and third-party widgets. ■cite■turn0search0■turn0search3■



3.2 Syntax

General form

```
useEffect(() => {
  // setup / synchronization
}, [dependencies])
```

3.3 Timer

Effect with cleanup

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds((current) => current + 1)
  }, 1000)

  return () => {
    clearInterval(id)
  }
}, [])
```

3.4 Event listener

Browser API synchronization

```
useEffect(() => {
  function handleResize() {
    console.log(window.innerWidth)
  }

  window.addEventListener(
    "resize",
    handleResize
  )

  return () => {
    window.removeEventListener(
      "resize",
      handleResize
    )
  }
}, [])
```

3.5 Dependency cases

No dependency array

```
// After every render
useEffect(() => {
  console.log("rendered")
})
```

Empty array

```
// Initial mount / cleanup
useEffect(() => {
  console.log("mounted")
}, [])
```

Specific dependency

```
// When roomId changes
useEffect(() => {
  connect(roomId)

  return () => disconnect(roomId)
}, [roomId])
```

Do not memorize Effects as timing tricks

Ask: “What external system am I synchronizing with?” If the answer is “nothing”, first check whether the calculation can simply happen during render.

3.6 Fetching data with an Effect

API fetch

```
import { useEffect, useState } from "react"

function Users() {
  const [users, setUsers] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState("")

  useEffect(() => {
    async function load() {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        )

        if (!response.ok) {
          throw new Error("Request failed")
        }

        const data = await response.json()
        setUsers(data)
      } catch (err) {
        setError(err.message)
      } finally {
        setLoading(false)
```

API fetch (continued)

```
      }
    }

    load()
  }, [])

  if (loading) return <p>Loading...</p>
  if (error) return <p>Error: {error}</p>

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  )
}
```

3.7 AbortController cleanup

Cancelable request

```
useEffect(() => {
  const controller = new AbortController()

  async function load() {
    try {
      const response = await fetch(url, {
        signal: controller.signal
      })

      const data = await response.json()
      setData(data)
    } catch (err) {
      if (err.name !== "AbortError") {
        setError(err.message)
      }
    }
  }

  load()

  return () => controller.abort()
}, [url])
```

MODULE 3 • PART 4

Derived Data & Lifting State

4.1 Derived values

If a value can be calculated from existing props/state, it usually does not need its own state variable. Duplicate state creates synchronization problems.

Derived total

```
function Cart({ items }) {
  const total = items.reduce(
    (sum, item) =>
      sum + item.price * item.quantity,
    0
  )

  return <p>Total: ■{total}</p>
}
```

Duplicate state pattern

```
// Usually unnecessary
const [items, setItems] = useState([])
const [total, setTotal] = useState(0)

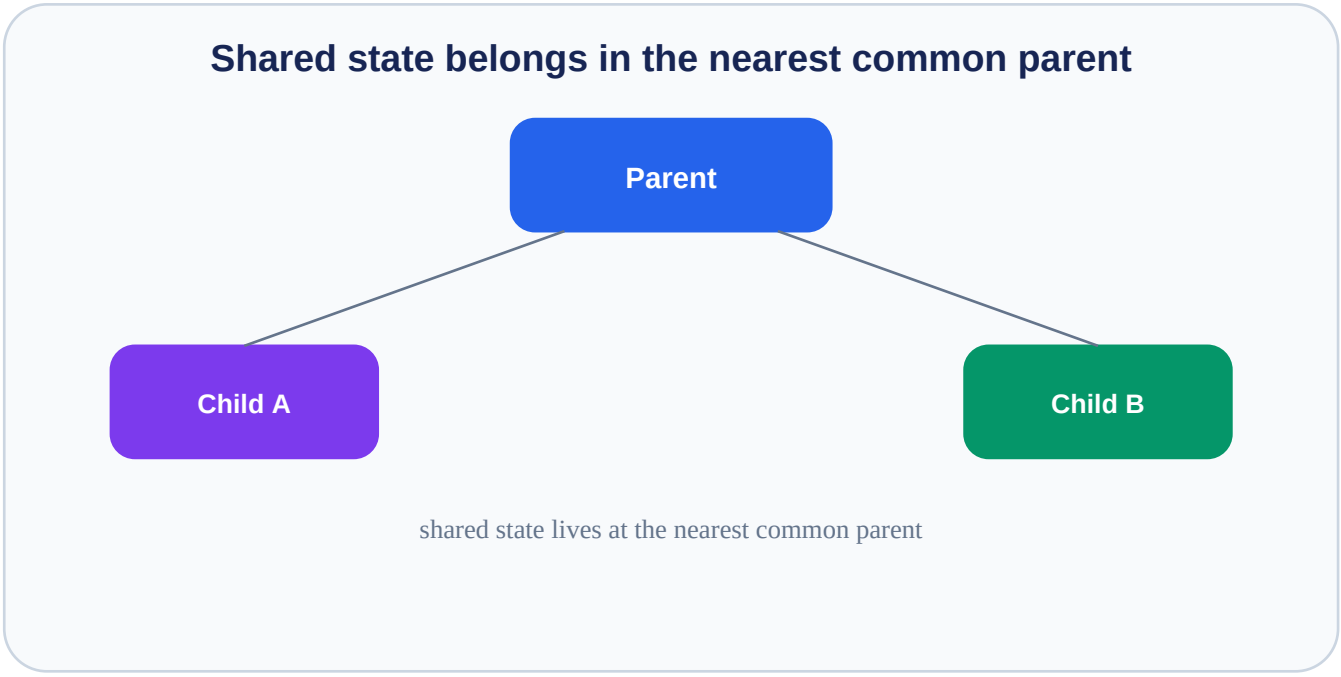
useEffect(() => {
  setTotal(calculateTotal(items))
}, [items])
```

4.2 Derived filtering

Derived search result

```
const visibleStudents = students.filter(
  (student) =>
    student.name
      .toLowerCase()
      .includes(search.toLowerCase())
)
```

4.3 Lifting state



When sibling components need the same information, move the state into their nearest common parent. The parent passes data down and callbacks down so children can request changes.

Lifted state

```
function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <Display count={count} />

      <Controls
        onIncrement={() =>
          setCount((c) => c + 1)
        }
      />
    </>
  )
}

function Display({ count }) {
  return <h1>{count}</h1>
}

function Controls({ onIncrement }) {
  return (
    <button onClick={onIncrement}>
      Increment
```

Lifted state (continued)

```
    </button>
  )
}
```

4.4 Callback prop

Child requests parent update

```
function Parent() {
  const [name, setName] = useState("")

  return (
    <Child
      onNameChange={setName}
    />
  )
}

function Child({ onNameChange }) {
  return (
    <input
      onChange={(event) =>
        onNameChange(event.target.value)
      }
    />
  )
}
```

Exercises: derived data & lifting

Exercise 5: Searchable List

Level: Intermediate

- Keep the full array in App.
- Keep search text in App.
- Pass search to a SearchBox.
- Filter the list as derived data.
- Do not store filtered results separately.

Exercise 6: Shared Counter

Level: Intermediate

- Create Display and Controls.
- Keep count in App.
- Pass count to Display.
- Pass increment/decrement/reset callbacks to Controls.

Solutions: lifting state

Solution 5

Solution

```
function App() {
  const [search, setSearch] = useState("")

  const visibleStudents = students.filter(
    (student) =>
      student.name
        .toLowerCase()
        .includes(search.toLowerCase())
  )

  return (
    <>
      <SearchBox
        value={search}
        onChange={setSearch}
      />

      <StudentList students={visibleStudents} />
    </>
  )
}

function SearchBox({ value, onChange }) {
  return (
```

Solution (continued)

```
    <input
      value={value}
      onChange={(event) =>
        onChange(event.target.value)
      }
    />
  )
}
```

Solution 6

Solution

```
function App() {
  const [count, setCount] = useState(0)

  return (
    <main>
      <Display count={count} />

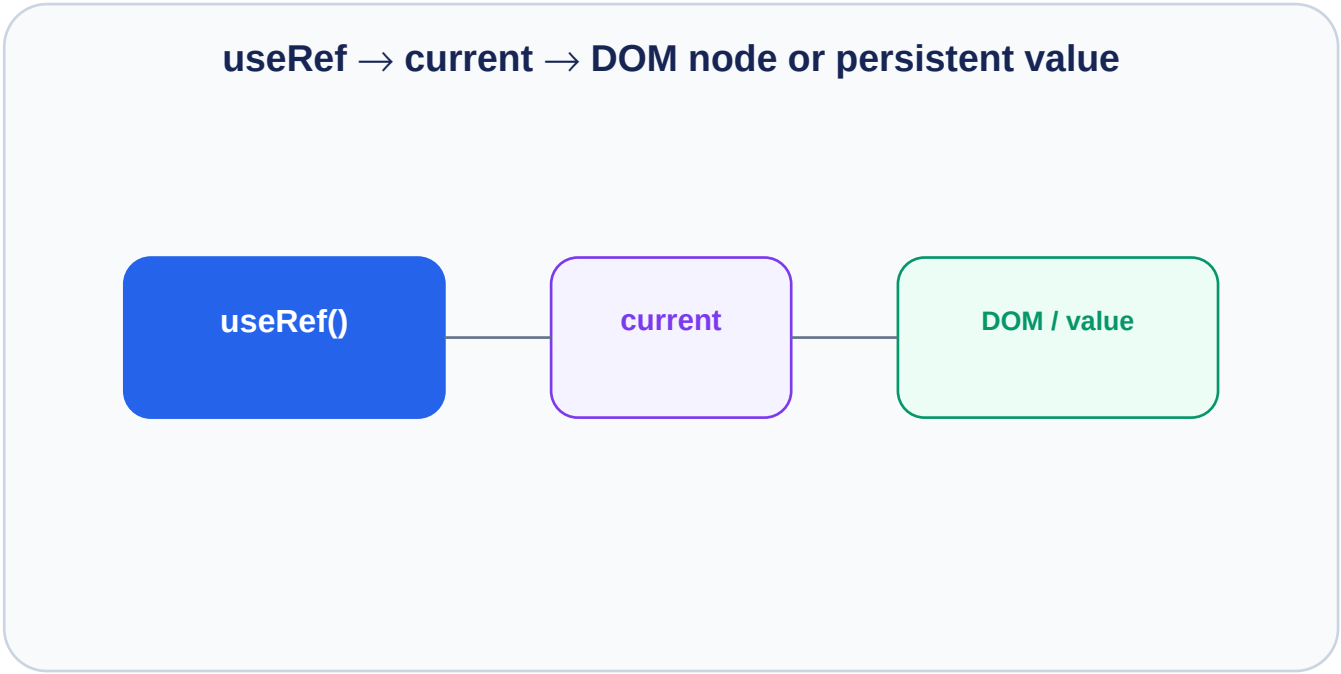
      <Controls
        onIncrement={() =>
          setCount((c) => c + 1)
        }
        onDecrement={() =>
          setCount((c) => c - 1)
        }
        onReset={() => setCount(0)}
      />
    </main>
  )
}
```

MODULE 3 • PART 5

useRef: DOM Access & Non-Rendering Values

5.1 What is useRef?

useRef returns a stable object with a current property. Changing ref.current does not cause a re-render. It is commonly used for DOM references and values that must persist without driving visible UI.
■cite■turn0search2■turn0search5■



5.2 Focus an input

DOM ref

```
import { useRef } from "react"

function Search() {
  const inputRef = useRef(null)

  function focusInput() {
    inputRef.current.focus()
  }

  return (
    <>
      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>
    </>
  )
}
```

5.3 State vs ref

Property	useState	useRef
Persists between renders	Yes	Yes
Change triggers render	Yes	No

Good for visible UI	Yes	Usually no
DOM node	Not its primary use	Common

5.4 Store an interval ID

Persistent non-rendering value

```
function Stopwatch() {
  const intervalRef = useRef(null)

  function start() {
    intervalRef.current = setInterval(() => {
      console.log("tick")
    }, 1000)
  }

  function stop() {
    clearInterval(intervalRef.current)
  }

  return (
    <>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </>
  )
}
```

5.5 Previous value pattern

Previous value

```
function PreviousValue({ value }) {
  const previous = useRef(value)

  useEffect(() => {
    previous.current = value
  }, [value])

  return (
    <p>
      Current: {value}
      <br />
      Previous: {previous.current}
    </p>
  )
}
```

Important

If a value should appear on screen and update the screen, use state. If changing it should not cause rendering, a ref may be appropriate. React explicitly notes that ref changes do not trigger re-renders. ■cite■turn0search2■

Exercise 7: Focus Manager

Exercise 7: Focus Manager

Level: Intermediate

- Create three inputs with three refs.
- Create buttons that focus the matching input.
- Use useRef(null).
- Do not put DOM elements in state.

Solution 7

Solution

```
function FocusManager() {
  const firstRef = useRef(null)
  const secondRef = useRef(null)
  const thirdRef = useRef(null)

  return (
    <div>
      <input ref={firstRef} />
      <button
        onClick={() => firstRef.current.focus()}
      >
        Focus first
      </button>

      <input ref={secondRef} />
      <button
        onClick={() => secondRef.current.focus()}
      >
        Focus second
      </button>

      <input ref={thirdRef} />
      <button
        onClick={() => thirdRef.current.focus()}
```

Solution (continued)

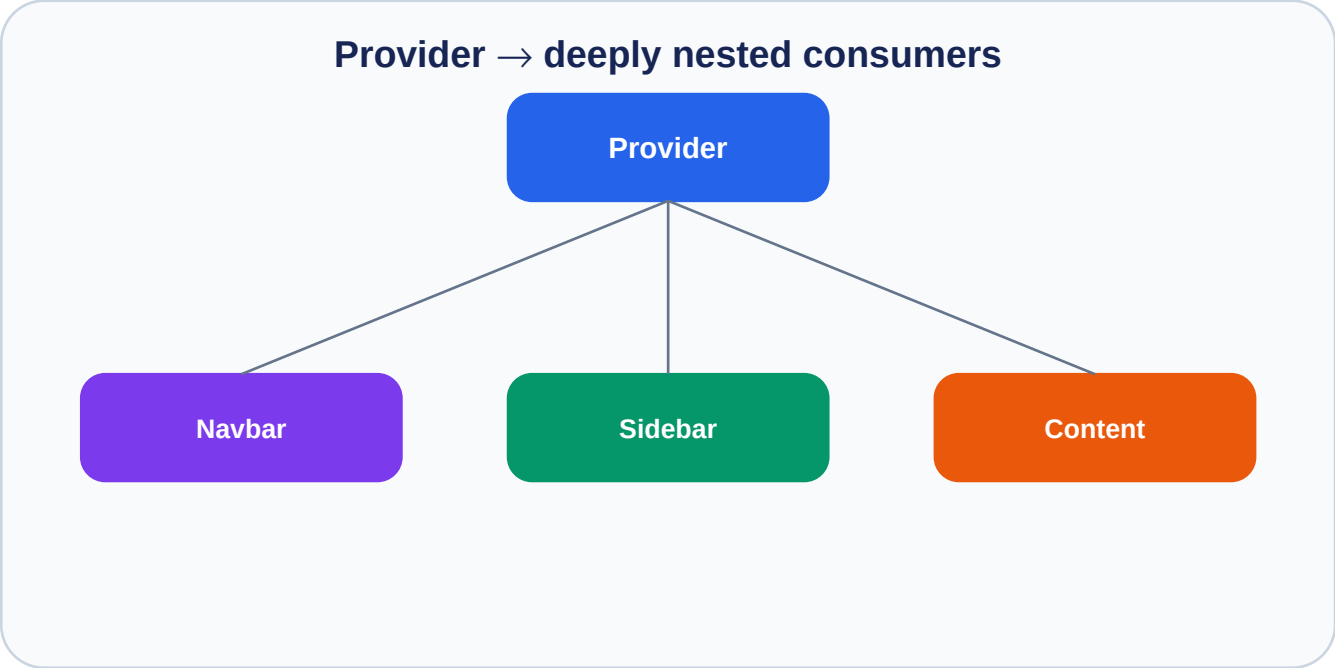
```
      >
        Focus third
      </button>
    </div>
  )
}
```

MODULE 3 • PART 6

useContext: Shared Data Across Deep Trees

6.1 Why Context?

Context is useful when a value is needed by many components at different depths. useContext reads the value from the closest matching provider above the component. ■cite■turn0search1■



6.2 Create context

ThemeContext.jsx

```
import { createContext } from "react"

export const ThemeContext =
  createContext("light")
```

6.3 Provide context

Provider

```
import { ThemeContext } from "./ThemeContext"

function App() {
  return (
    <ThemeContext value="dark">
      <Page />
    </ThemeContext>
  )
}
```

6.4 Read context

Consumer

```
import { useContext } from "react"
import { ThemeContext } from "./ThemeContext"

function Button() {
  const theme = useContext(ThemeContext)

  return (
    <button>
      Current theme: {theme}
    </button>
  )
}
```

6.5 Context + state

Context with state

```
const ThemeContext = createContext(null)

function App() {
  const [theme, setTheme] = useState("light")

  return (
    <ThemeContext value={{ theme, setTheme }}>
      <Page />
    </ThemeContext>
  )
}

function ThemeButton() {
  const { theme, setTheme } =
    useContext(ThemeContext)

  return (
    <button
      onClick={() =>
        setTheme((current) =>
          current === "light"
            ? "dark"
            : "light"
        )
```

Context with state (continued)

```
      }
    >
      {theme}
    </button>
  )
}
```

Do not make everything global

Context is useful for shared concerns such as theme or authenticated user information. Local component state should remain local when possible.

6.6 Context directory

Directory

```
src/
■■■ context/
■   ■■■ ThemeContext.jsx
■■■ components/
■   ■■■ Navbar.jsx
■   ■■■ ThemeButton.jsx
■■■ App.jsx
■■■ main.jsx
```

Exercise 8: Theme Context

Exercise 8: Theme Context

Level: Intermediate+

- Create ThemeContext.
- Keep theme in App state.
- Provide theme and toggleTheme.
- Read it from Navbar and Card.
- Build light/dark visual states.

Solution 8

Solution

```
const ThemeContext = createContext(null)

function App() {
  const [theme, setTheme] = useState("light")

  const value = {
    theme,
    toggleTheme: () =>
      setTheme((current) =>
        current === "light"
          ? "dark"
          : "light"
      )
  }

  return (
    <ThemeContext value={value}>
      <Navbar />
      <Card />
    </ThemeContext>
  )
}

function Navbar() {
```

Solution (continued)

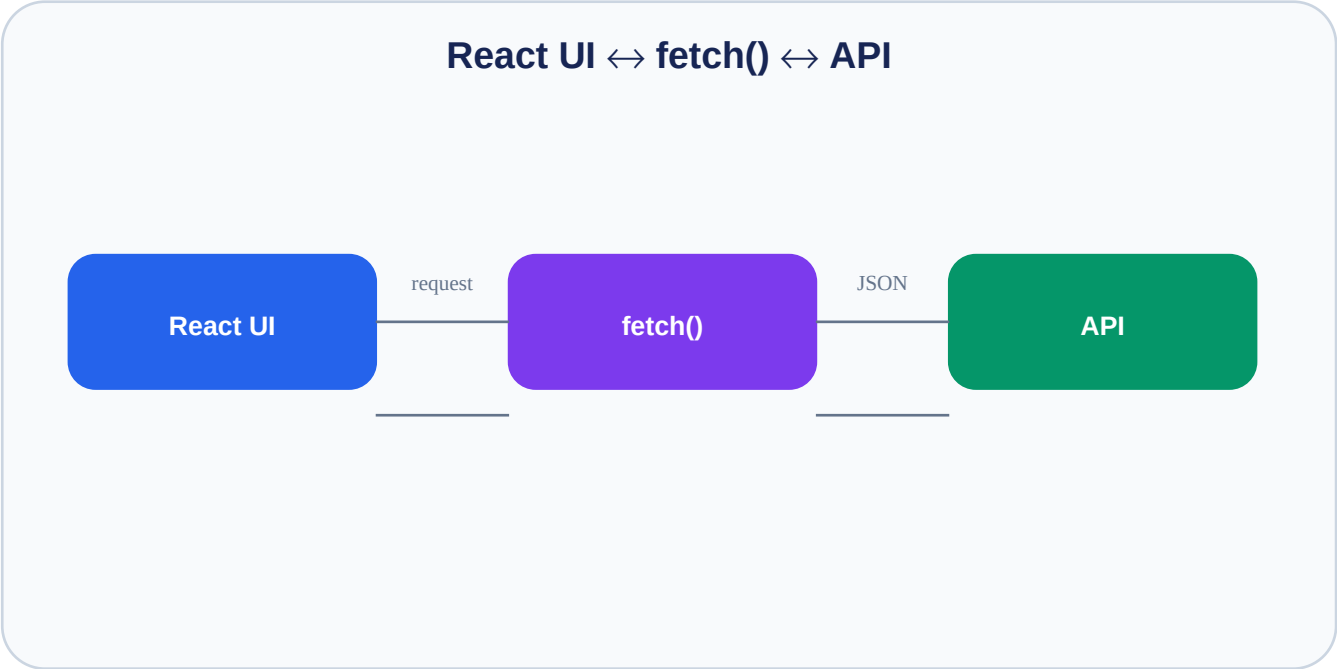
```
  const { theme, toggleTheme } =
    useContext(ThemeContext)

  return (
    <nav>
      <span>{theme}</span>
      <button onClick={toggleTheme}>
        Toggle
      </button>
    </nav>
  )
}
```

MODULE 3 • PART 7

API Data: Loading, Error, Empty & Success

7.1 Four states of network UI



- **Loading:** request is in progress.
- **Success:** data arrived and can be rendered.
- **Error:** request or parsing failed.
- **Empty:** request succeeded but there is nothing to show.

7.2 Full fetch pattern

Complete API state example

```
function Users() {
  const [users, setUsers] = useState([])
  const [status, setStatus] = useState("idle")
  const [error, setError] = useState("")

  useEffect(() => {
    const controller = new AbortController()

    async function load() {
      setStatus("loading")

      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users",
          { signal: controller.signal }
        )

        if (!response.ok) {
          throw new Error("Request failed")
        }

        const data = await response.json()

        setUsers(data)
```

Complete API state example (continued)

```
        setStatus("success")
      } catch (err) {
        if (err.name === "AbortError") return

        setError(err.message)
        setStatus("error")
      }
    }

    load()

    return () => controller.abort()
  }, [])

  if (status === "loading") {
    return <p>Loading...</p>
  }

  if (status === "error") {
    return <p>Error: {error}</p>
  }

  if (status === "success" && users.length === 0) {
    return <p>No users found.</p>
```

Complete API state example (continued)

```
  }

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  )
}
```

7.3 Search-based request

Search request

```
const [query, setQuery] = useState("")
const [results, setResults] = useState([])

useEffect(() => {
  if (!query.trim()) {
    setResults([])
    return
  }

  const controller = new AbortController()

  async function search() {
    const response = await fetch(
      `/api/search?q=${encodeURIComponent(query)}`,
      { signal: controller.signal }
    )

    const data = await response.json()
    setResults(data)
  }

  search()

  return () => controller.abort()
```

Search request (continued)

```
}, [query])
```

7.4 API architecture

Recommended separation

```
src/
■■■ components/
■   ■■■ SearchForm.jsx
■   ■■■ Results.jsx
■   ■■■ Loading.jsx
■   ■■■ ErrorMessage.jsx
■■■ hooks/
■   ■■■ useUsers.js
■■■ App.jsx
■■■ main.jsx
```

MODULE 3 • PART 8

Custom Hooks: Reusing Stateful Logic

8.1 Why custom Hooks?

A custom Hook extracts reusable stateful logic. Each component that calls the Hook gets its own instance of that logic unless the Hook connects to shared state such as Context.

useToggle.js

```
import { useState } from "react"

export function useToggle(initial = false) {
  const [value, setValue] = useState(initial)

  function toggle() {
    setValue((current) => !current)
  }

  return {
    value,
    toggle
  }
}
```

Using the Hook

```
function Menu() {
  const { value: open, toggle } =
    useToggle(false)

  return (
    <>
      <button onClick={toggle}>
        {open ? "Close" : "Open"}
      </button>

      {open && <nav>Menu</nav>}
    </>
  )
}
```

8.2 useLocalStorage

Reusable persistence Hook

```
import { useEffect, useState } from "react"

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key)

    return saved
      ? JSON.parse(saved)
      : initialValue
  })

  useEffect(() => {
    localStorage.setItem(
      key,
      JSON.stringify(value)
    )
  }, [key, value])

  return [value, setValue]
}
```

8.3 useWeather custom Hook

Custom API Hook

```
export function useWeather(city) {
  const [data, setData] = useState(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState("")

  useEffect(() => {
    if (!city.trim()) return

    const controller = new AbortController()

    async function load() {
      setLoading(true)
      setError("")

      try {
        const response = await fetch(
          `/api/weather?city=${encodeURIComponent(city)}`,
          { signal: controller.signal }
        )

        if (!response.ok) {
          throw new Error("Could not load weather")
        }
```

Custom API Hook (continued)

```
        setData(await response.json())
      } catch (err) {
        if (err.name !== "AbortError") {
          setError(err.message)
        }
      } finally {
        setLoading(false)
      }
    }

    load()

    return () => controller.abort()
  }, [city])

  return { data, loading, error }
}
```

Hook naming

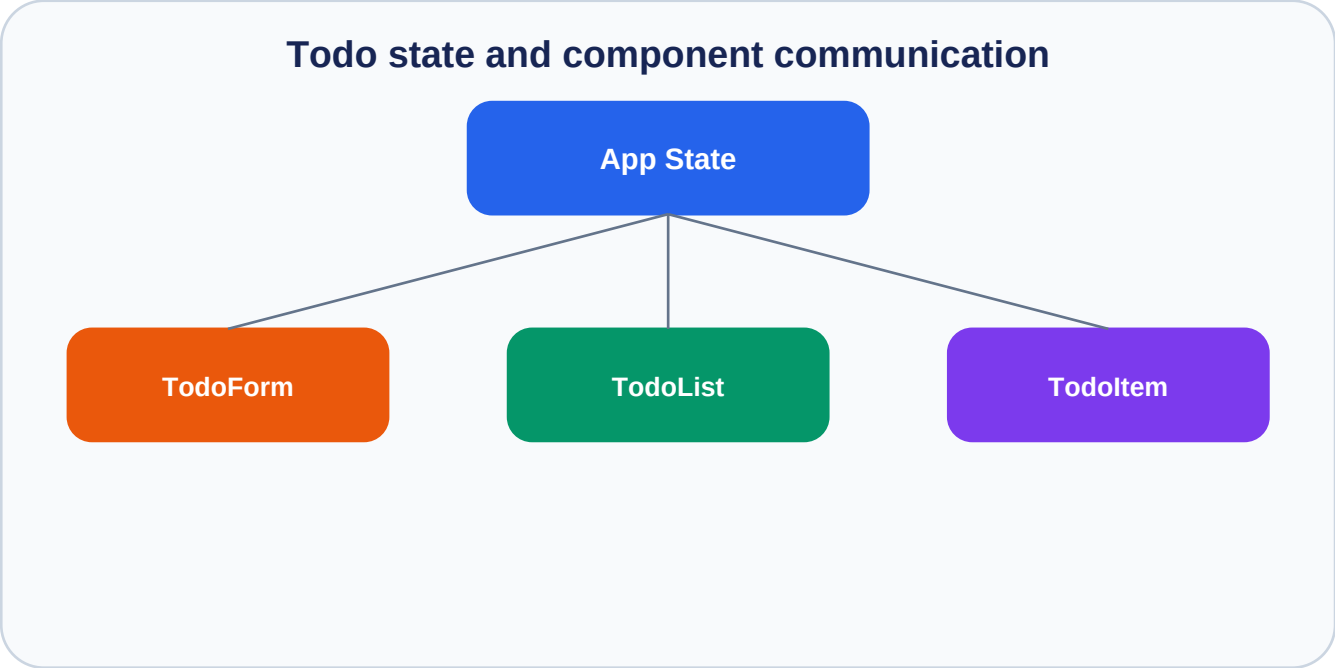
Custom Hooks normally start with use. That naming convention makes their Hook usage obvious and lets React's tooling recognize Hook patterns.

MODULE 3 • PART 9

Project 1: Complete Todo Application

9.1 Project goal

This project combines state, forms, props, derived filtering, immutable updates and component communication into one application.



9.2 Directory

Project structure

```
src/
■■■ components/
■   ■■■ TodoForm.jsx
■   ■■■ TodoList.jsx
■   ■■■ TodoItem.jsx
■   ■■■ FilterButtons.jsx
■■■ App.jsx
■■■ App.css
■■■ index.css
■■■ main.jsx
```

9.3 Add todo

App logic

```
function addTodo(text) {
  setTodos((current) => [
    ...current,
    {
      id: crypto.randomUUID(),
      text,
      completed: false
    }
  ])
}
```

9.4 Toggle & delete

Immutable actions

```
function toggleTodo(id) {
  setTodos((current) =>
    current.map((todo) =>
      todo.id === id
        ? {
            ...todo,
            completed: !todo.completed
          }
        : todo
    )
  )
}

function deleteTodo(id) {
  setTodos((current) =>
    current.filter((todo) => todo.id !== id)
  )
}
```

9.5 TodoForm

TodoForm.jsx

```
function TodoForm({ onAdd }) {
  const [text, setText] = useState("")

  function handleSubmit(event) {
    event.preventDefault()

    if (!text.trim()) return

    onAdd(text.trim())
    setText("")
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={text}
        onChange={(event) =>
          setText(event.target.value)
        }
      />

      <button type="submit">
        Add
      </button>
```

TodoForm.jsx (continued)

```
    </form>
  )
}
```

9.6 TodoItem

TodoItem.jsx

```
function TodoItem({
  todo,
  onToggle,
  onDelete
}) {
  return (
    <li>
      <label>
        <input
          type="checkbox"
          checked={todo.completed}
          onChange={() => onToggle(todo.id)}
        />

        <span className={
          todo.completed
            ? "todo todo--done"
            : "todo"
        }>
          {todo.text}
        </span>
      </label>

      <button
```

TodoItem.jsx (continued)

```
        onClick={() => onDelete(todo.id)}
      >
        Delete
      </button>
    </li>
  )
}
```

9.7 Derived filter

Do not store filteredTodos in state

```
const filteredTodos = todos.filter((todo) => {
  if (filter === "active") {
    return !todo.completed
  }

  if (filter === "completed") {
    return todo.completed
  }

  return true
})
```

9.8 Complete Todo App

Complete App.jsx

```
function App() {
  const [todos, setTodos] = useState([])
  const [filter, setFilter] = useState("all")

  function addTodo(text) {
    setTodos((current) => [
      ...current,
      {
        id: crypto.randomUUID(),
        text,
        completed: false
      }
    ])
  }

  function toggleTodo(id) {
    setTodos((current) =>
      current.map((todo) =>
        todo.id === id
          ? { ...todo, completed: !todo.completed }
          : todo
      )
    )
  }
```

Complete App.jsx (continued)

```
  function deleteTodo(id) {
    setTodos((current) =>
      current.filter((todo) => todo.id !== id)
    )
  }

  const filteredTodos = todos.filter((todo) => {
    if (filter === "active") return !todo.completed
    if (filter === "completed") return todo.completed
    return true
  })

  return (
    <main>
      <h1>Todo App</h1>

      <TodoForm onAdd={addTodo} />

      <FilterButtons
        filter={filter}
        onChange={setFilter}
      />
```

Complete App.jsx (continued)

```
      <TodoList
        todos={filteredTodos}
        onToggle={toggleTodo}
        onDelete={deleteTodo}
      />
    </main>
  )
}
```

9.9 Todo App: Pure CSS

App.css

```
.todo-app {
  max-width: 680px;
  margin: 40px auto;
  padding: 24px;
}

.todo-form {
  display: flex;
  gap: 10px;
}

.todo-form input {
  flex: 1;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
}

.todo-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 14px;
  margin-top: 10px;
```

App.css (continued)

```
  border: 1px solid #e2e8f0;
  border-radius: 12px;
  background: white;
}

.todo--done {
  color: #94a3b8;
  text-decoration: line-through;
}
```

9.10 Todo App: Tailwind

Tailwind layout

```
<main className="mx-auto max-w-2xl p-6">
  <h1 className="mb-6 text-3xl font-bold">
    Todo App
  </h1>

  <form className="flex gap-2">
    <input className="
      min-w-0 flex-1 rounded-lg
      border border-slate-300 px-3 py-2
    " />

    <button className="
      rounded-lg bg-blue-600
      px-4 py-2 font-bold text-white
    ">
      Add
    </button>
  </form>
</main>
```

MODULE 3 • PART 10

Project 2: API Dashboard

10.1 Goal

Build an API-driven dashboard that searches for a resource and shows loading, error, empty and success states. The provider can be swapped later; the React architecture is the lesson.

10.2 Directory

Project structure

```
src/
■■■ components/
■   ■■■ SearchForm.jsx
■   ■■■ ResultCard.jsx
■   ■■■ Loading.jsx
■   ■■■ ErrorMessage.jsx
■■■ hooks/
■   ■■■ useSearch.js
■■■ App.jsx
■■■ main.jsx
```

10.3 Search Hook

useSearch.js

```
export function useSearch(query) {
  const [data, setData] = useState([])
  const [status, setStatus] = useState("idle")
  const [error, setError] = useState("")

  useEffect(() => {
    if (!query.trim()) {
      setData([])
      setStatus("idle")
      return
    }

    const controller = new AbortController()

    async function load() {
      setStatus("loading")

      try {
        const response = await fetch(
          `/api/search?q=${encodeURIComponent(query)}`,
          { signal: controller.signal }
        )

        if (!response.ok) {
```

useSearch.js (continued)

```
          throw new Error("Request failed")
        }

        const result = await response.json()
        setData(result)
        setStatus("success")
      } catch (err) {
        if (err.name === "AbortError") return

        setError(err.message)
        setStatus("error")
      }
    }

    load()

    return () => controller.abort()
  }, [query])

  return { data, status, error }
}
```

10.4 SearchForm

SearchForm.jsx

```
function SearchForm({
  value,
  onChange,
  onSubmit
}) {
  return (
    <form onSubmit={onSubmit}>
      <input
        value={value}
        onChange={(event) =>
          onChange(event.target.value)
        }
        placeholder="Search..."
      />

      <button type="submit">
        Search
      </button>
    </form>
  )
}

export default SearchForm
```

10.5 App states

App.jsx

```
function App() {
  const [input, setInput] = useState("")
  const [query, setQuery] = useState("")

  const { data, status, error } =
    useSearch(query)

  function handleSubmit(event) {
    event.preventDefault()
    setQuery(input.trim())
  }

  return (
    <main>
      <SearchForm
        value={input}
        onChange={setInput}
        onSubmit={handleSubmit}
      />

      {status === "loading" && (
        <p>Loading...</p>
      )}
```

App.jsx (continued)

```
      {status === "error" && (
        <p>{error}</p>
      )}

      {status === "success" && data.length === 0 && (
        <p>No results.</p>
      )}

      {status === "success" && data.length > 0 && (
        <ResultList data={data} />
      )}
    </main>
  )
}
```

Module 3 Practical Exercises

Exercise 9: Timer Hook

Level: Advanced

- Create useTimer.
- Return seconds, running, start, stop and reset.
- Use an Effect with cleanup.
- Build a Timer component using the Hook.

Exercise 10: API User List

Level: Intermediate+

- Fetch users from a JSON API.
- Show loading, error and success states.
- Show an empty message when the array is empty.
- Cancel the request during cleanup.

Exercise 11: Theme Context

Level: Intermediate+

- Create ThemeContext.
- Keep theme in state.
- Expose theme and toggleTheme.
- Use the context from two deeply nested components.

Exercise 12: Persistent Username

Level: Advanced

- Build useLocalStorage.
- Store a username.
- Refresh the page and confirm the value persists.
- Add a Clear button.

Exercise Solutions

Solution 9: useTimer

Solution

```
export function useTimer() {
  const [seconds, setSeconds] = useState(0)
  const [running, setRunning] = useState(false)

  useEffect(() => {
    if (!running) return

    const id = setInterval(() => {
      setSeconds((current) => current + 1)
    }, 1000)

    return () => clearInterval(id)
  }, [running])

  return {
    seconds,
    running,
    start: () => setRunning(true),
    stop: () => setRunning(false),
    reset: () => setSeconds(0)
  }
}
```

Solution 10: API state model

State machine idea

```
const [data, setData] = useState([])
const [status, setStatus] = useState("idle")
const [error, setError] = useState("")

// idle → loading → success
// idle → loading → error
// success + [] → empty UI
```

Solution 11: Theme Context

Solution

```
const ThemeContext = createContext(null)

function App() {
  const [theme, setTheme] = useState("light")

  const value = {
    theme,
    toggleTheme() {
      setTheme((current) =>
        current === "light"
          ? "dark"
          : "light"
      )
    }
  }

  return (
    <ThemeContext value={value}>
      <Layout />
    </ThemeContext>
  )
}

function Layout() {
```

Solution (continued)

```
  return (
    <main>
      <Navbar />
      <Content />
    </main>
  )
}

function Navbar() {
  const { theme, toggleTheme } =
    useContext(ThemeContext)

  return (
    <nav>
      <span>{theme}</span>
      <button onClick={toggleTheme}>
        Toggle
      </button>
    </nav>
  )
}
```

Solution 12: useLocalStorage

Solution

```
export function useLocalStorage(
  key,
  initialValue
) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key)

    return saved
      ? JSON.parse(saved)
      : initialValue
  })

  useEffect(() => {
    localStorage.setItem(
      key,
      JSON.stringify(value)
    )
  }, [key, value])

  return [value, setValue]
}
```

Debugging Module 3

State changes but screen does not

Why: A normal variable was changed instead of state.

Fix: Use useState and its setter for visible state.

Array update behaves strangely

Why: The existing array was mutated.

Fix: Use map/filter/spread to create a new array.

Object field disappeared

Why: The whole object was replaced with a partial object.

Fix: Spread the previous object before changing one property.

Effect runs too often

Why: A dependency changes or the Effect is doing work that should be derived during render.

Fix: Identify the external system and inspect dependencies.

Timer duplicates

Why: An Effect creates a timer without cleanup or reconnects repeatedly.

Fix: Return clearInterval() from the Effect.

Ref changed but UI stayed the same

Why: That is expected. Ref changes do not trigger renders.

Fix: Use state if the value drives UI.

Context is undefined

Why: The component is not under the correct provider or the provider value is missing.

Fix: Inspect the component tree and value prop.

API shows stale data

Why: An older request completed after a newer request.

Fix: Use AbortController or another request identity strategy.

Choosing the Right React Tool

Need	Use	Example
Visible value changes	useState	Counter
Parent supplies data	Props	ProductCard title
Many deep components need same value	Context	Theme
DOM node / non-rendering value	useRef	Focus input
External system synchronization	useEffect	Timer/API/browser listener
Can calculate from existing data	Derived value	Cart total

The big question

Before adding another Hook, ask what problem you are solving. React becomes easier when every piece of state or synchronization has a clear reason to exist.

Module 3 Final Assessment: Course Manager

Build a Course Manager where an instructor can add, search, filter, edit status and delete courses. This should be the first project where students must reason about architecture instead of just following a recipe.

Requirements

- Store courses in state.
- Create AddCourseForm, CourseList, CourseCard, FilterBar and Stats components.
- Use controlled form fields.
- Add, delete and toggle courses.
- Search by title.
- Filter All/Active/Inactive.
- Calculate total/active/inactive counts as derived values.
- Use stable IDs.
- Persist courses with useLocalStorage.
- Use one legitimate Effect for synchronization.
- Use Context for a theme or other clearly shared concern.
- Use responsive CSS or Tailwind.
- Show loading/error UI if an API feature is added.

Suggested directory

Target structure

```
src/
■■■ components/
■   ■■■ AddCourseForm.jsx
■   ■■■ CourseCard.jsx
■   ■■■ CourseList.jsx
■   ■■■ FilterBar.jsx
■   ■■■ Stats.jsx
■■■ hooks/
■   ■■■ useLocalStorage.js
■■■ context/
■   ■■■ ThemeContext.jsx
■■■ App.jsx
■■■ App.css
■■■ index.css
■■■ main.jsx
```

Final Assessment: Data Model

Course object

```
const initialCourses = [
  {
    id: crypto.randomUUID(),
    title: "React Fundamentals",
    instructor: "Riya",
    duration: "6 weeks",
    active: true
  },
  {
    id: crypto.randomUUID(),
    title: "JavaScript",
    instructor: "Aman",
    duration: "8 weeks",
    active: true
  }
]
```

Recommended state

Minimal state

```
const [courses, setCourses] =
  useLocalStorage("courses", initialCourses)

const [search, setSearch] = useState("")
const [filter, setFilter] = useState("all")
```

Derived values

Derived UI

```
const filteredCourses = courses.filter((course) => {
  const matchesSearch =
    course.title
      .toLowerCase()
      .includes(search.toLowerCase())

  const matchesFilter =
    filter === "all" ||
    (filter === "active" && course.active) ||
    (filter === "inactive" && !course.active)

  return matchesSearch && matchesFilter
})

const stats = {
  total: courses.length,
  active: courses.filter((c) => c.active).length,
  inactive: courses.filter((c) => !c.active).length
}
```

Architecture checkpoint

Do not create state for filteredCourses, total, active and inactive. Those values already exist conceptually inside courses and can be calculated.

Module 3 Mastery Checklist

Students should be able to complete these without copying the examples:

- Explain state in their own words.
- Use useState with strings, numbers, booleans, arrays and objects.
- Update arrays without mutation.
- Update objects without mutation.
- Use functional state updates.
- Explain why state is different from a normal variable.
- Explain when an Effect is appropriate.
- Write cleanup for timers/listeners/requests.
- Calculate derived values instead of storing duplicates.
- Lift state to a common parent.
- Pass callback props.
- Use useRef to access a DOM element.
- Explain why changing ref.current does not render.
- Create and consume Context.
- Fetch API data with explicit UI states.
- Create a custom Hook.
- Build the Todo app from scratch.
- Build the Course Manager assessment.

Oral revision questions

- Why does setState cause a render?
- Why should state arrays usually be replaced instead of mutated?
- When is the functional updater useful?
- What is an Effect actually synchronizing with?
- Why does an Effect need cleanup for timers and listeners?
- What is derived state?
- Why do we lift state?
- When is useRef better than useState?
- What problem does Context solve?
- What are loading, success, error and empty states?

Instructor Teaching Plan

#	Topic	Time	Focus
1	State basics	45–60 min	Counter + variable vs state
2	useState	60 min	Primitive, arrays, objects, updater
3	useEffect	60–90 min	Timer + browser event + API
4	Derived/lifting	60 min	Search + shared counter
5	useRef	45 min	Focus + persistent value
6	useContext	60 min	Theme context
7	API data	60–90 min	Loading/error/success/empty
8	Custom Hooks	60 min	useToggle + persistence
9	Todo project	90–120 min	Build without solution first
10	API project	90–120 min	Hook + dashboard
11	Assessment	120 min	Course Manager

Teaching sequence

Explain → type a tiny example → deliberately break it → debug → refactor into a component/Hook → exercise → solution. This produces understanding rather than ceremonial copy-paste.

Official React References

This module's Hook behavior was cross-checked against the current official React documentation.

- React Hooks reference: built-in state, effect, ref and context Hooks. ■cite■turn0search3■
- useEffect: synchronization with external systems and cleanup. ■cite■turn0search0■
- useRef: persistent values and DOM references without automatic re-rendering. ■cite■turn0search2■turn0search5■
- useContext: reading values from the nearest provider. ■cite■turn0search1■

End of Module 3

Students have now moved from static component composition to stateful, asynchronous and shared-data applications. Module 4 can build on this with routing, larger application architecture, data layers and production-style project organization.